



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A02_CPE Caso practico extendido – Prof. Emilio
Ramirez**

Autor: Segura Abad, Mario

Fecha: 22 de Febrero, 2026

INTRODUCCIÓN..... 2

ENUNCIADO 3

 Trabajo con promesas: acceso a ficheros en NodeJS: acceso a ficheros en NodeJS 4

DESARROLLO DE LA PRÁCTICA..... 4

 Descripción..... 5

 • Leer, añadir, actualizar y borrar gastos almacenados en un fichero JSON. 5

 • Practicar el uso de promesas encadenadas con fs.readFile y fs.writeFile. 5

 • Comprender cómo manipular datos JSON en NodeJS. 5

 • Superar los tests automáticos proporcionados en el proyecto. 5

 Contenido del archivo 5

 Caso Práctico Extendido U04_A02_CPE - Librería de gestión de gastos con promesas 5

 Objetivo:..... 5

 Claves de resolución:..... 6

 Cómo visualizarlo 6

 Requisitos: 6

 • Node.js..... 6

 • Visual Studio Code..... 6

 • Terminal integrada 6

 Pasos: 6

 1. Descargar el proyecto y abrirlo en Visual Studio Code, en mi caso..... 6

 2. Ejecutar: npm install..... 6

 3. Completar las funciones marcadas con TODO..... 6

 4. Ejecutar los tests: npm run test 6

 5. Revisar la consola para:..... 6

 6. Ver los resultados de los tests..... 6

 7. Ver mensajes de autoría. 6

 8. Abrir el fichero datos.json para comprobar que los cambios se guardan correctamente. 6

 Objetivos de la práctica..... 6

 • Comprender el uso de promesas en NodeJS. 6

 • Manipular ficheros JSON mediante fs.readFile y fs.writeFile. 6

 • Encadenar tareas asíncronas correctamente. 6

 • Gestionar errores en operaciones de lectura y escritura. 6

 • Documentar el caso práctico extendido con capturas de pantalla y fuentes..... 6

CONCLUSIÓN..... 7

INTRODUCCIÓN

En este ejercicio accederemos a ficheros en NodeJS mediante las promesas. Deberemos descargar todos los archivos del proyecto para poder completar el caso práctico.

ENUNCIADO

Trabajo con promesas: acceso a ficheros en NodeJS: acceso a ficheros en NodeJS

Caso práctico que ejemplifica los contenidos.

Descargad los archivos de proyecto para completar el caso práctico extendido.

Ejemplo de implementación:

Captura de pantalla – accesoDatos.js (prueba de autoría)

```
1 // Autor: Mario Segura Abad
2 // Fecha: 11/02/2026
3
```

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 4 Apartado 2

Descripción

Este apartado se centra en el **Caso Práctico Extendido U04_A02_CPE**, cuyo objetivo es crear una **librería Node.js** capaz de gestionar gastos de distintos usuarios utilizando **promesas** y la API `fs` del propio NodeJS.

La aplicación no usaba bases de datos: todos los datos se almacenan en un fichero `datos.json`.

El objetivo principal ha sido:

- Leer, añadir, actualizar y borrar gastos almacenados en un fichero JSON.
- Practicar el uso de promesas encadenadas con `fs.readFile` y `fs.writeFile`.
- Comprender cómo manipular datos JSON en NodeJS.
- Superar los tests automáticos proporcionados en el proyecto.

Contenido del archivo

Caso Práctico Extendido U04_A02_CPE - Librería de gestión de gastos con promesas

Objetivo:

Crear una librería que gestione los gastos de distintos usuarios almacenados en `datos.json`, implementando cuatro funciones que devuelvan promesas:

1. **obtenerGastosUsuario(usuario)**
 - Lee el fichero.
 - Convierte el JSON a objeto.
 - Devuelve el array de gastos del usuario o uno vacío si no existe.
2. **anyadirGastoUsuario(usuario, gasto)**
 - Lee el fichero.
 - Convierte el JSON a objeto.
 - Añade el gasto al usuario (creándolo si no existe).
 - Escribe el fichero actualizado.
 - Devuelve la promesa de `writeFile`.
3. **actualizarGastoUsuario(usuario, gastold, nuevosDatos)**
 - Lee el fichero.
 - Convierte el JSON a objeto.
 - Busca el gasto por id y lo actualiza.
 - Si no existe usuario o gasto --> lanza error.
 - Escribe el fichero actualizado.
4. **borrarGastoUsuario(usuario, gastold)**
 - Lee el fichero.

- Convierte el JSON a objeto.
- Elimina el gasto indicado.
- Si no existe usuario --> lanza error.
- Escribe el fechero actualizado.

Claves de resolución:

- Descargar el proyecto base y revisar los archivos marcados con **TODO**.
- Instalar dependencias del proyecto: `npm install`
- Ejecutar los tests: `npm run test`
- Recordar que:
 - `readFile` y `writeFile` devuelven promesas.
 - Todas las funciones deben devolver una promesa.
 - `then()` devuelve otra promesa, por lo que permite encadenar tareas.
 - `writeFile` solo admite texto --> es obligatorio usar `JSON.stringify`.
- Comprobar que los datos se guardan con la estructura correcta.

Ejemplo de ejecución:

Captura de pantalla - Instrucción o comando para realizar el test

```
PS C:\Users\mario\GitHub\DWEC-2-DAW\TEMA 4 - Programación asíncrona\EJERCICIOS-TAREAS\U04_A02_CPE Caso práctico extendido\DWEC_U04_A02_CPE_E archivos> npm run test

> promesas_ficheros@1.0.0 test
> mocha
```

Salida por consola – accesoDatos.js (comprobación de las funciones)

Comprobación de las funciones de acceso a datos

- ✓ Función 'obtenerGastosUsuario'
- ✓ Función 'anyadirGastoUsuario'
- ✓ Función 'actualizarGastoUsuario': usuario y gasto existen.
- ✓ Función 'actualizarGastoUsuario': usuario no existe.
- ✓ Función 'borrarGastoUsuario': usuario y gasto existen.
- ✓ Función 'borrarGastoUsuario': usuario no existe.

6 passing (28ms)

Cómo visualizarlo

Requisitos:

- Node.js
- Visual Studio Code
- Terminal integrada

Pasos:

1. Descargar el proyecto y abrirlo en Visual Studio Code, en mi caso.
2. Ejecutar: `npm install`
3. Completar las funciones marcadas con TODO.
4. Ejecutar los tests: `npm run test`
5. Revisar la consola para:
6. Ver los resultados de los tests.
7. Ver mensajes de autoría.
8. Abrir el fichero `datos.json` para comprobar que los cambios se guardan correctamente.

Objetivos de la práctica

- Comprender el uso de promesas en NodeJS.
- Manipular ficheros JSON mediante `fs.readFile` y `fs.writeFile`.
- Encadenar tareas asíncronas correctamente.
- Gestionar errores en operaciones de lectura y escritura.
- Documentar el caso práctico extendido con capturas de pantalla y fuentes.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de **promesas en NodeJS** y el acceso a ficheros mediante la API `fs`.

Asimismo, hemos aprendido a leer, modificar y escribir datos en formato JSON, así como a gestionar errores y encadenar operaciones asíncronas.

El ejercicio ha sido clave para entender cómo funcionan las operaciones de E/S en NodeJS y cómo estructurar una librería modular basada en promesas.